

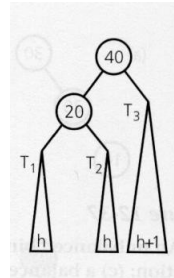
AVL Trees

Objectives

- To know what an AVL tree is.
- To understand how to rebalance a tree using the LL rotation, LR rotation, RR rotation, and RL rotation.
- To know how to design the AVLTree class.
- To insert elements into an AVL tree.
- To implement node rebalancing.
- To delete elements from an AVL tree.
- To implement the AVLTree class.
- To test the AVLTree class.
- To analyze the complexity of search, insert, and delete operations in AVL trees.

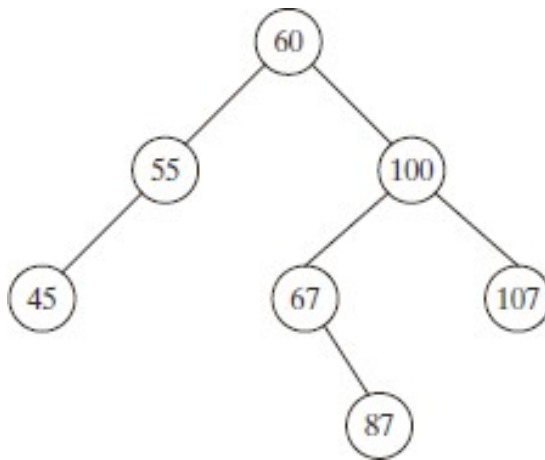
AVL Trees

- AVL Trees are Binary Search Trees with the additional notion of “height” for subtrees.



- Height information can be represented as a member field of the Node class.

Heights of each node



Height of any node can be computed by adding 1 to the height of the taller of the left or right subtrees.

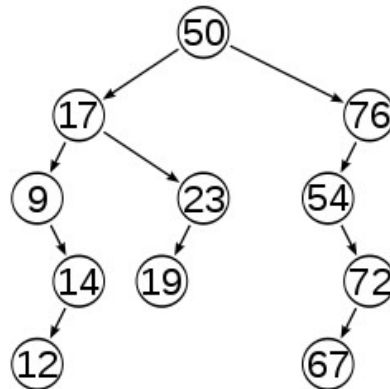
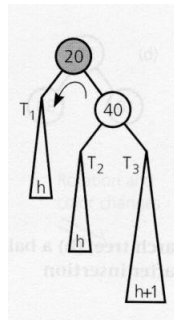
Height of node 45, 55, 60 ?

Restrictions

- The only rule then is:
 - No node is allowed to have children who differ in height by more than one.

Example: when children differ in height by at least 2

An example of an unbalanced tree: which nodes are unbalanced ?

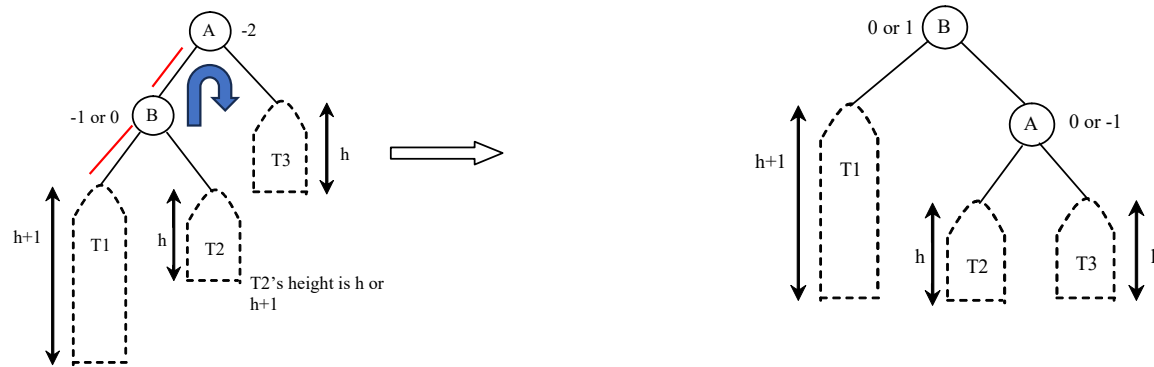


Balance Factor/Left-Heavy/Right-Heavy

The process for inserting or deleting an element in an AVL tree is the same as in a regular binary search tree. The difference is that you may have to rebalance the tree after an insertion or deletion operation. The *balance factor* of a node is the height of its right subtree minus the height of its left subtree. A node is said to be *balanced* if its balance factor is -1, 0, or 1. A node is said to be *left-heavy* if its balance factor is -1. A node is said to be *right-heavy* if its balance factor is +1. A node is said to be imbalanced if its balance factor is -2 or + 2

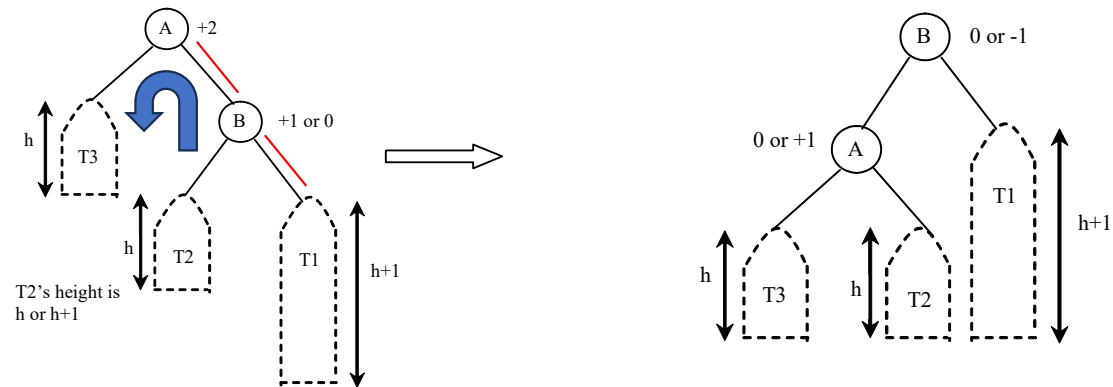
Left-Heavy Imbalance (LL imbalance) LL Rotation

Right Rotation: A LL **imbalance** occurs at node A such that A has a balance factor -2 and a left child B with a balance factor -1 or 0 . This type of imbalance can be fixed by performing a single right rotation at A.



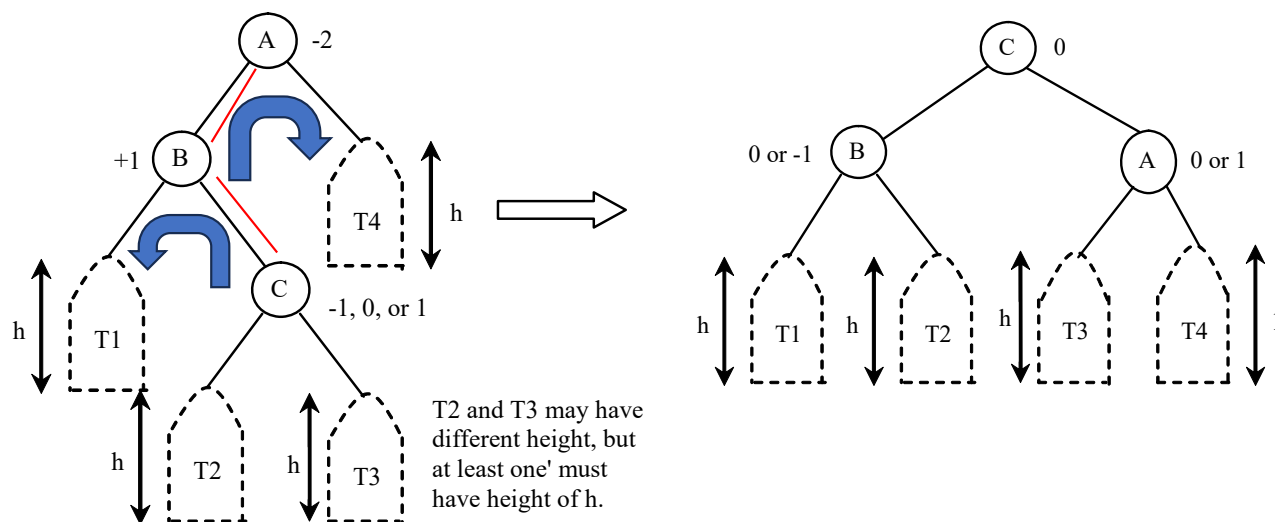
Right-Heavy Imbalance (RR imbalance) and RR Rotation

Left Rotation: A **RR imbalance** occurs at a node A such that A has a balance factor +2 and a right child B with a balance factor +1 or 0. This type of imbalance can be fixed by performing a single left rotation at A.



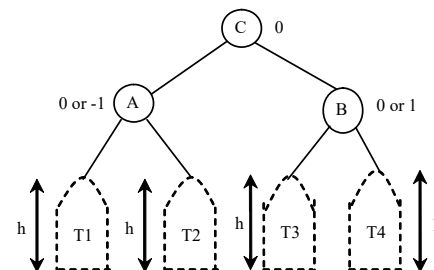
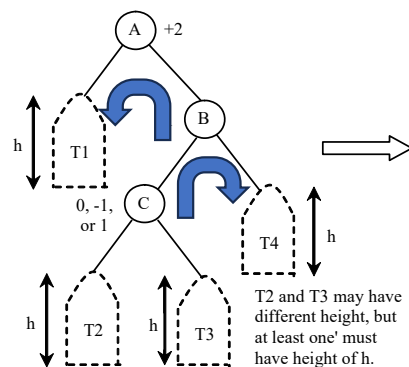
LR imbalance and LR rotation

LR Rotation: An *LR imbalance* occurs at a node A such that A has a balance factor -2 and a left child B with a balance factor +1. Assume B's right child is C. This type of imbalance can be fixed by performing a double rotation (first a single left rotation at B and then a single right rotation at A).



RL Imbalance and RL Rotation

RL Rotation: An **RL imbalance** occurs at a node A such that A has a balance factor +2 and a right child B with a balance factor -1 . Assume B's left child is C. This type of imbalance can be fixed by performing a double rotation (first a single right rotation at B and then a single left rotation at A).



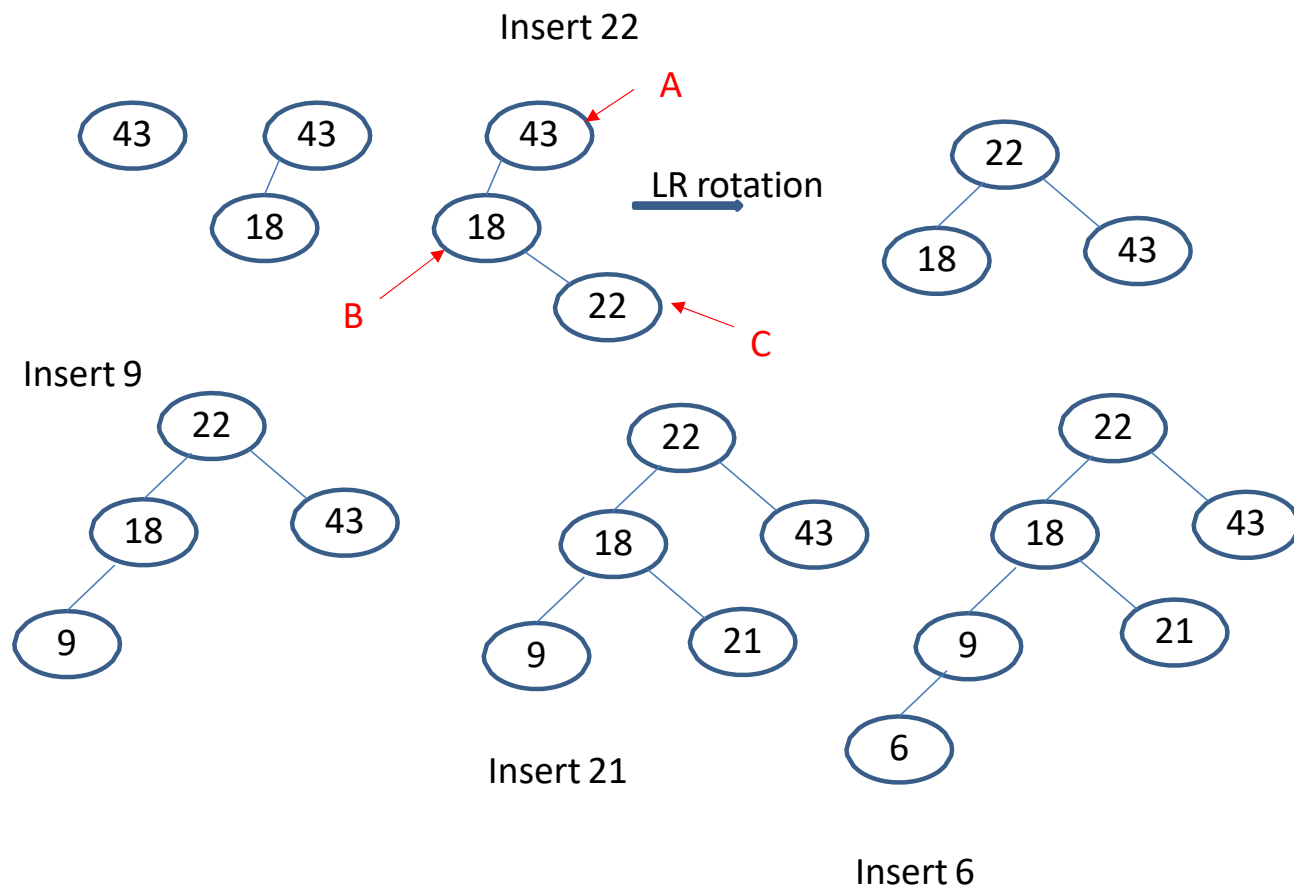
Insertion

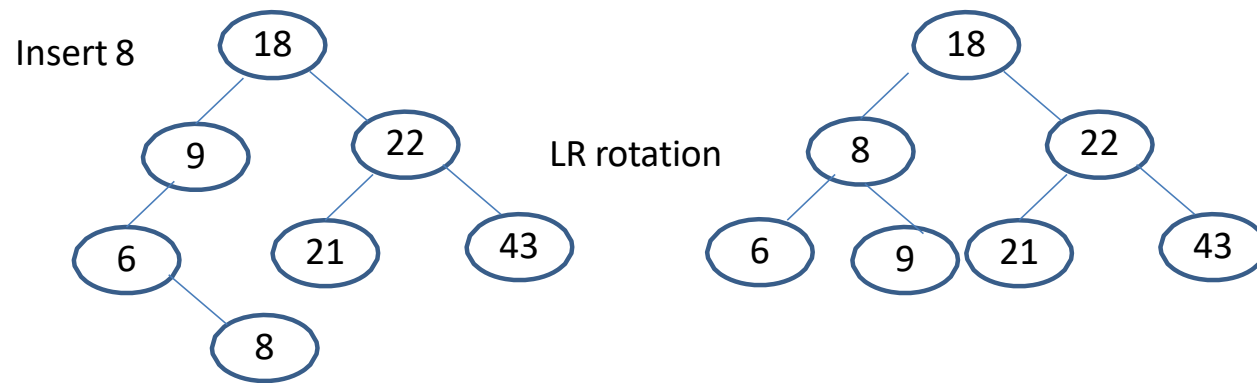
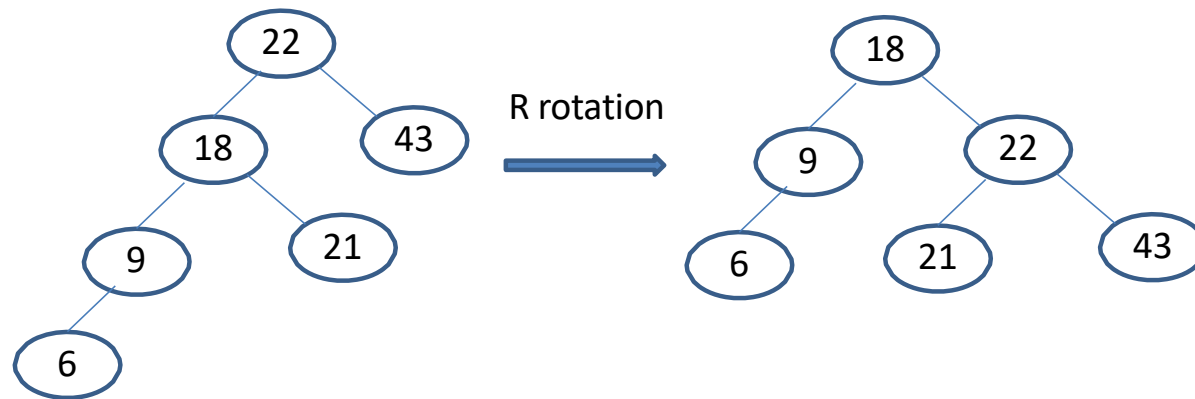
Algorithm of insertion

- Insert the new node as your insert in a BS Tree
- Check heights of nodes along the path **from the new leaf to the root.**
- If found one unbalanced node => rebalance from that node

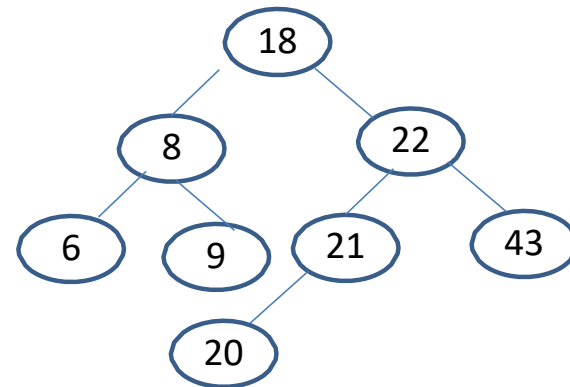
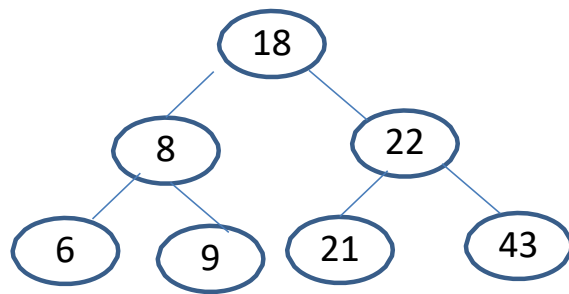
An example

Insert 43,18,22,9,21,6,8,20,63,50

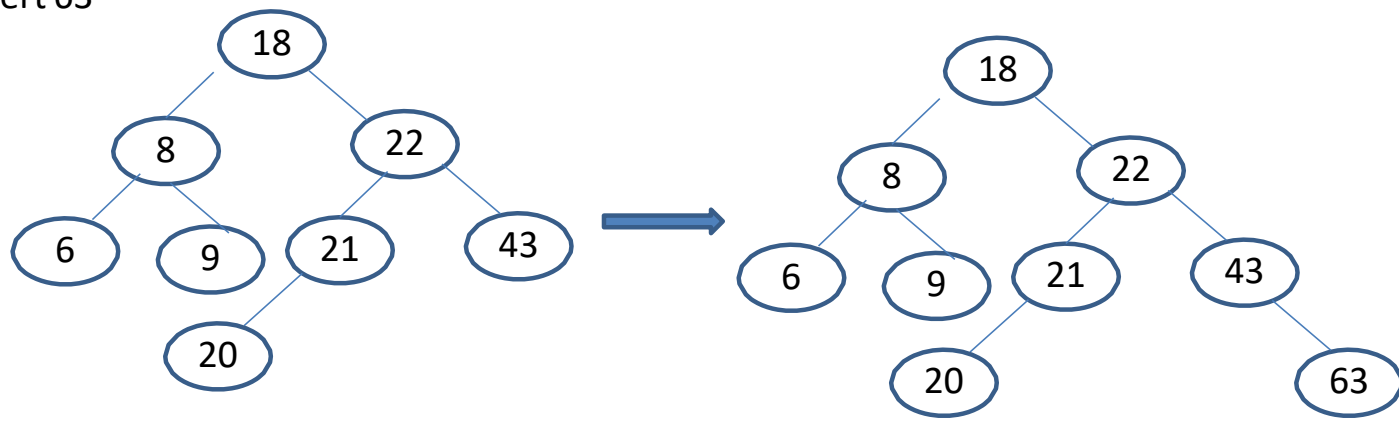




Insert 20

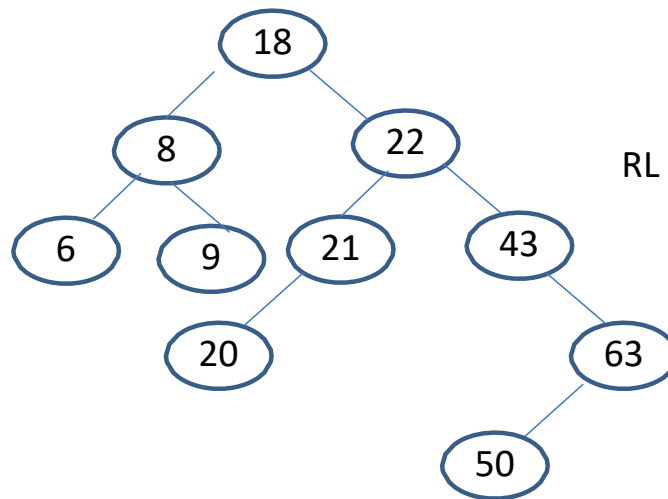


Insert 63

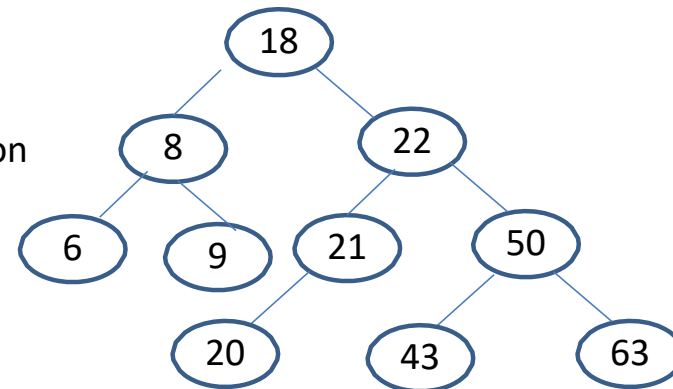


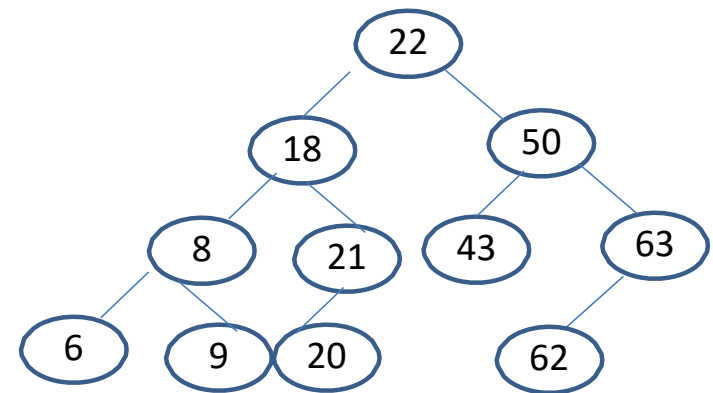
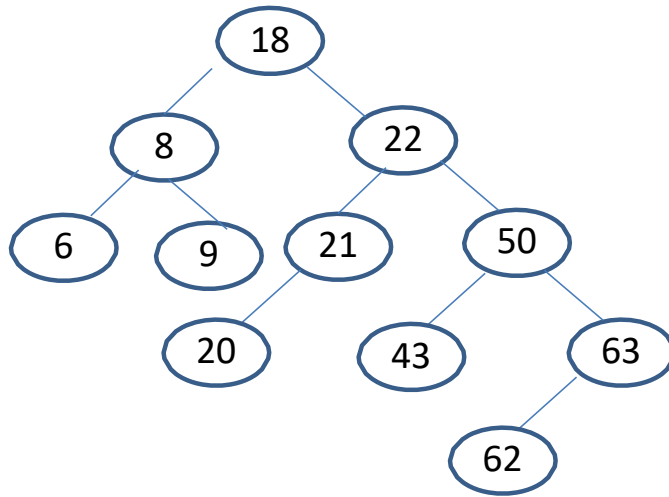
More nodes ?
One team volunteer 62

Insert 50

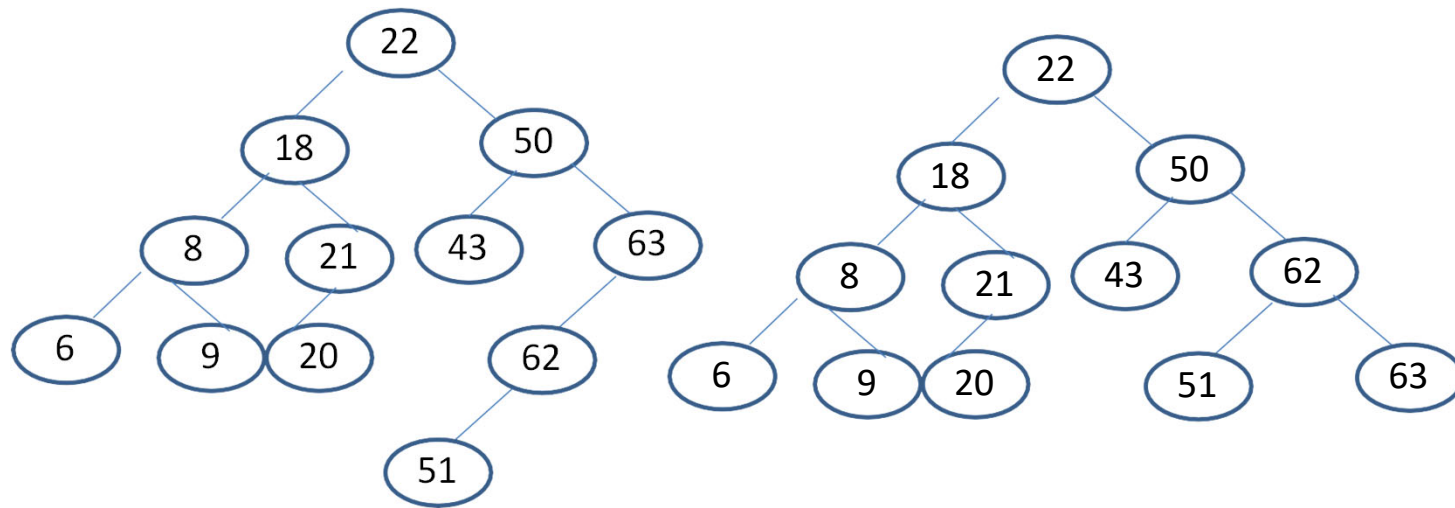


RL rotation





One more node
51 ?



An in class exercise

1,2,3,4,5,6,7,8 ?

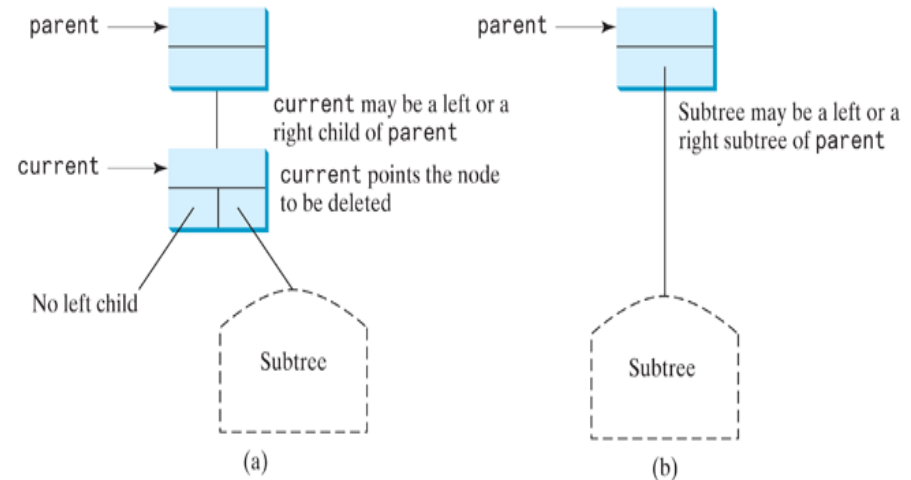
More difficult one

Insert 14, 17, 11, 7, 53, 4, 13, 12 and 8

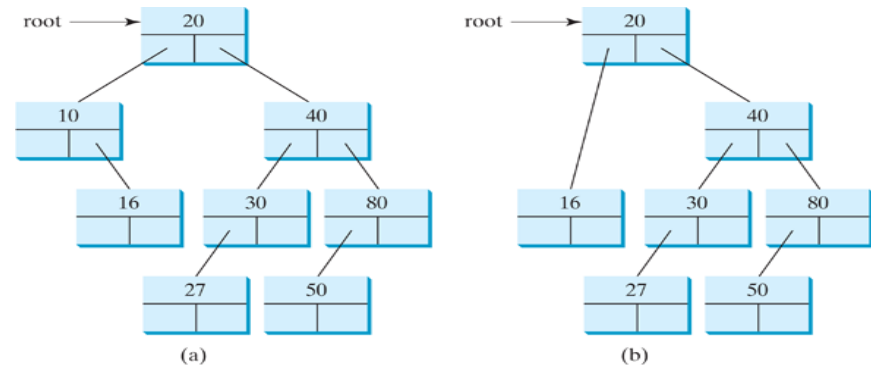
Deleting: AVLTree

- Deleting is same as deleting from a BST, except the tree need to be rebalanced.
- Locate the node to be deleted.
 - Let **current** point that contains the element to be deleted.
 - Let **parent** point to the parent of the **current**.
- Two cases:

Case 1: The **current** does not have a left child ([Figure](#)). Simply connect the **parent** node with the right child of the **current** node.

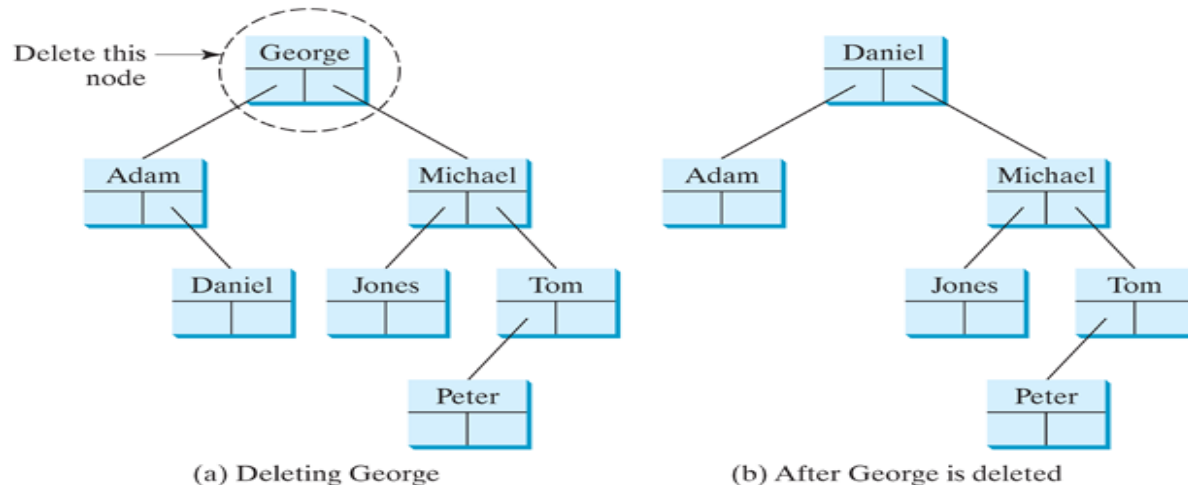


Delete 10

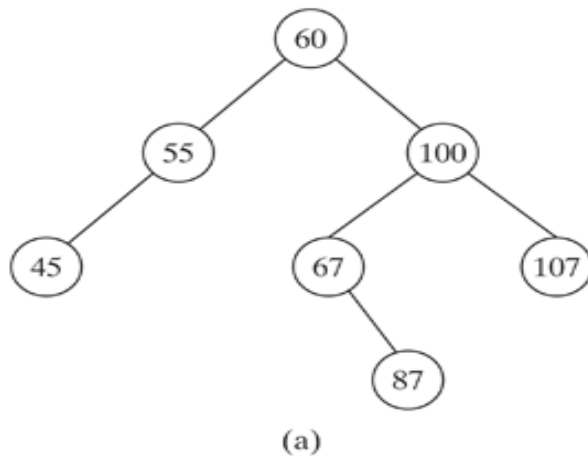


Case 2: the **current** has a left child.

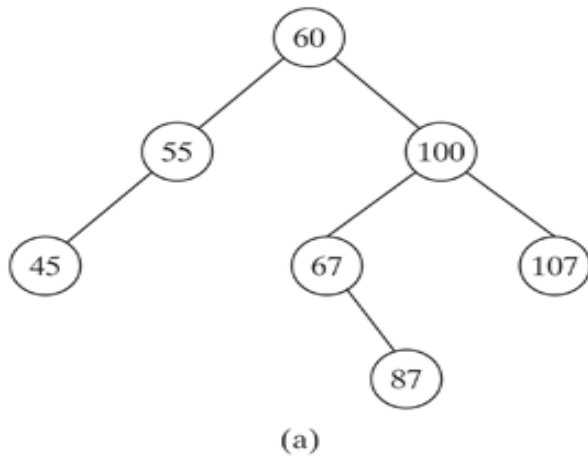
- Let the **rightMost** point to node that contains the largest element in the left subtree of the **current** node and **parentOfRightMost** point to the parent node of the **rightMost** node ([Figure a](#)).
- The **rightMost** node can not have a right child, but may have a left child. Replace the element value in the **current** node with the one in the **rightMost** node, connect the **parentOfRightMost** node with left child of the **rightMost** node, and delete the **rightMost**. ([Figure b](#))



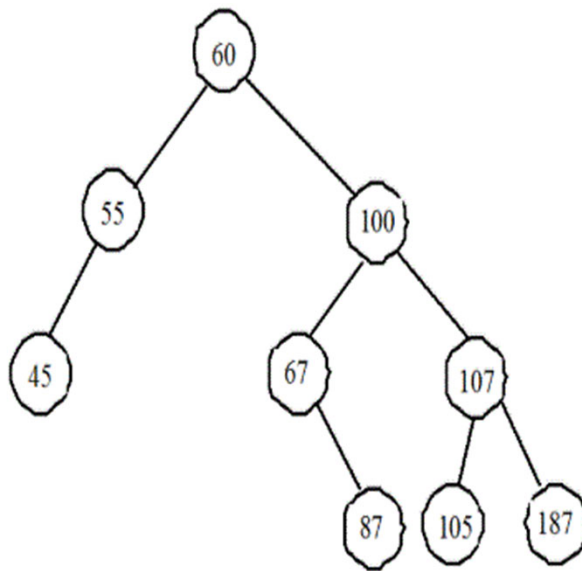
Example 1: delete 107 from (a). What rotation do you perform to rebalance the tree? Which node was unbalanced?



Example 2: delete 55 from (a). What rotation do you perform to rebalance the tree? Which node was unbalanced?



Example 3: For the AVL tree in (b), show the new AVL tree after deleting elements 67 and 87. What rotation did you perform in order to rebalance the tree? Which node was unbalanced?



(b)

Designing Classes for AVL Trees

An AVL tree is a binary tree. So you can define the AVLTree class to extend the BST class.

